

Reference : <https://www.thymeleaf.org/doc/tutorials/2.1/usingthymeleaf.html>

What is Thymeleaf?

The Thymeleaf is an open-source Java library licensed under the Apache License 2.0. It is a HTML5/XHTML/XML template engine.

It is a server-side Java template engine for web (servlet-based) environments. It provides full integration with Spring Framework.

It is based on XML tags and attributes. These XML tags define the execution of predefined logic on the DOM (Document Object Model) It is a substitute for JSP.

The architecture of Thymeleaf allows the fast processing of templates that depends on the caching of parsed files. It uses the least possible amount of I/O operations during execution.

Unlike JSP , it does not get compiled into a servlet.

1. Create a spring boot project , with additional dependency of thymeleaf
2. Default location of thymeleaf templates is `<src>/main/resources/templates`
Can also be added under subfolders or can be replaced by any other folder.

3. Create a new HTML under `<templates>`

index.html

4. Add XML namespace for thymeleaf.

```
<html xmlns:th="http://www.thymeleaf.org">
```

5. To display , model attributes

```
<h4 th:text="'some text' + ${model attribute name} "></h4>
```

6. `<a th:href="@{/order/list}">List Orders</a>`

If our app is installed at <http://localhost:8080/myapp>, after clicking on this link :
<http://localhost:8080/myapp/order/list>

- 7.

Switch case

```
<div th:switch="${user.role}">
  <p th:case="'admin'">User is an administrator</p>
  <p th:case="#{roles.manager}">User is a manager</p>
  <p th:case="*">Some other User role</p>
</div>
```

8. Links with dynamic attributes

For adding a query string , request param

eg : `<a th:href="@{/students/edit(id=${student.id})}">Edit</a>`

Will produce : <http://localhost:8080/students/edit?id=1>

9. For adding a path variable :

```
<a th:href="@{/students/delete/{id}(id=${student.id})}">Delete</a>
```

Will produce : <http://localhost:8080/students/delete/1>

10. Form Handling

eg :

```
<form action="#" th:action="@{/greeting}" th:object="${greeting}" method="post">
  <p>Id: <input type="text" th:field="*{id}" /></p>
  <p>Message: <input type="text" th:field="*{content}" /></p>
  <p><input type="submit" value="Submit" /> <input type="reset" value="Reset" /></p>
</form>
```

Meaning :

10.1 The `th:action="@{/greeting}"` expression submits the form to POST to the `http://host:port/greeting` endpoint

10.2 `th:object="${greeting}"` binds form data to model attribute by the name of greeting.

10.3 The two form fields, expressed with `th:field="*{id}"` and `th:field="*{content}"`, correspond to the properties of Greeting POJO

11. if-else

(there is no else here ,unless is a negation of if)

```
<td>
  <span th:if="${teacher.gender == 'F'}">Female</span>
  <span th:unless="${teacher.gender == 'F'}">Male</span>
</td>
```